INTER IIT
TECH MEET 15.0

HIGH PREP

rime

Voice for how India actually talks

INTRODUCTION

Rime builds speech models trained on how people actually talk, not on studio narration. The models are made for voice agents and for any other product that speaks. Real conversation is messy: people switch languages halfway through a sentence, say names and numbers in their own way, and pause, laugh and hesitate. A voice that serves India has to cope with all of that. This problem statement is about Indian voice technology in general, not just customer-service agents.

PROBLEM STATEMENT

Indian voice technology currently faces fundamental flaws. Most Indian languages lack clean, transcribed training data, and voice clones usually fail to maintain the same voice across different languages. Additionally, AI agents frequently misread typical Indian text inputs like Hinglish, IDs, and amounts, and there are no reliable automated metrics to judge speech quality without extensive human testing.

Because these failures are interconnected, a successful Indian voice system must speak the user's language, sound identical across languages, read localized text flawlessly, and be measurable without human raters. Your objective is to address a portion of this overarching problem, build a functional solution, and prove its efficacy through rigorous testing.

You must select one of two tracks. Track 1 requires building a four-part multilingual system that handles low-resource language training data, cross-lingual voice cloning, precise agent text reading, and an automated speech quality judge. Alternatively, Track 2 involves developing a speech captioner and a voice generator that mutually improve by learning from unlabelled Hindi and Tamil audio.

Both paths represent a research challenge prioritizing a focused, carefully tested, and honestly reported idea over a long list of features. Because designing a fair assessment is central to this work, you are also expected to create your own evaluation metrics

THE CHALLENGE

You are a small research team building Indian voice technology and showing that it works.

1. **Pick a track. Track 1 is a four-part voice system. Track 2 is a captioner and a** generator that improve each other. You tell the organizers on Discord which track you enter.
2. **Build against a baseline. Every part of your system is compared with a** public, reproducible baseline. You choose the strongest eligible one and get it approved by a judge on Discord.
3. **Test each other. At the mid-term, every team contributes test sets. Your** system is then evaluated on sets written and recorded by other teams, by the organizers' native listeners and by fixed measurement tools.
4. **Freeze, run, listen. You lock your system at the freeze.** You run it on the evaluation inputs and lock your outputs. The organizers run all the listening tests, and you score your locked outputs on the automatic measures. You hand in your final submission and present.

Each team enters one of the two tracks and tells the organizers on Discord. Both tracks share the dates, rules, scoring rubric and deliverables. Only the tasks and the results measures are different. All teams from both tracks are ranked together, and there will be overall winners.

In the shared sections, a "part" is one of the four parts of Track 1. In Track 2, a "part" is the captioner or the generator. Part numbers and the Part 1 language apply only to Track 1. In Track 2, the "rest of the system" in Scoring means the model that is not your lead part, plus the methods of your loop. These methods are how the loop targets weak areas, filters labels and adds ground truth from outside the loop.

KEY DATES

The organizers announce the dates of all milestones on Discord: mid-term, freeze, evaluation window, end-term and final presentation.

CHOOSING YOUR TRACK

Tell the organizers on Discord which track your team enters. Mentors for that track then join your team's group. Within one week after the release of this problem statement, also confirm these items on Discord:

- your lead part, and its contribution in one sentence;
- a scored baseline for each part;
- in Track 1, your Part 1 language.

A judge approves your baselines on Discord. If several teams plan the same approach for their lead part, the judges will tell them and may suggest alternatives. No team has to change its approach.

Your choices are binding once you confirm them:

- Your track can't change.
- You can change your Part 1 language or your lead part only before the mid-term submission, and only with a judge's approval on Discord. After a language change, every language must still have at least two teams.
- You can replace an approved baseline only with a stronger eligible one, before the freeze, and only with a judge's approval.

Support. Discord is the main support channel. Ask questions any time; judges answer within two working days.

TRACK 1: MULTILINGUAL INDIAN VOICE SYSTEM

THE PROBLEM

Track 1 takes on four connected failures of Indian voice technology at once: too little clean transcribed speech in most languages, cloned voices that can't speak their owner's other languages, agents that misread Hinglish, amounts, IDs and names, and no reliable way to measure quality without rooms full of raters. You build one voice that exists in the listener's language, sounds like the same person in every language, reads the text it is given correctly, and can be measured without a room of raters. You also do real speech research along the way.

THE CHALLENGE

Build one multilingual Indian voice system with four connected parts.

Part	What you build	Stage of the speech pipeline
1. Data from almost nothing	An automated pipeline that turns scattered, messy audio into training data for an underrepresented Indian language	Before training
2. My voice, my languages	One system that takes 30 seconds of someone speaking Hindi, Tamil or English, and speaks Hindi, Tamil, English and your Part 1 language in their voice	The model
3. Say it right	The same system reads raw agent text correctly (Hinglish, Tamil–English, amounts, IDs and names), switching language mid-sentence without changing voice	Text into speech
4. Judge without a jury	An automatic evaluator that scores naturalness, pronunciation and speaker similarity in Hindi and Tamil, and agrees with native listeners	After generation

The parts feed each other. Part 1's data teaches your Part 2 system a new language. Part 3 makes that system read real agent text in a cloned voice. Part 4 measures what it produces. We expect teams to split into sub-teams, one per part, and bring their work together into one system.

Choose a lead part. Every part must work: your frozen system must produce an output for every evaluation item in each part, and each part is measured against its own baseline. Within one week after the release of this problem statement, name one part on Discord as your lead research contribution. Novelty and depth are judged mainly on that part. The other parts need a working system, a fair baseline comparison and measured results. Novel work in the other parts is welcome and gets credit. The Scoring section gives the exact points.

Depth over completion. Results and research quality are scored separately. A focused idea in your lead part, tested carefully and followed by an honest look at what did and did not work, can score well on research quality even when its results are negative.

Optional extras. Each part lists optional directions that give your team its own angle. They count under novelty and depth, not results points, and they add no required work. If you claim something for them, back it up with a listening check or an automatic measure.

You build your own evaluation. At the mid-term, every team contributes a test set for each part, and every part is evaluated on sets contributed by other teams. In Part 1, these are held-out sentences written by other teams in your language, alongside samples of your pipeline's output and its baseline's. Teams freeze their systems before the final input release and random sampling. See How Evaluation Works.

Languages at a glance:

Part 1	One of Bhojpuri, Chhattisgarhi, Magahi or Maithili
Part 2	Hindi, Tamil and English, plus your Part 1 language
Part 3	Hindi–English and Tamil–English
Part 4	Hindi and Tamil

PART 1: DATA FROM ALMOST NOTHING

The problem. Public Indian speech datasets mostly cover the major languages. Bhojpuri, Chhattisgarhi, Magahi and Maithili are spoken by millions of people, but most of their recorded speech is noisy, spontaneous and rarely paired with accurate text.

Your task. Build an automated pipeline that turns your language's raw data pool into clean, aligned text–audio pairs that are ready for TTS training. Use its output to teach your Part 2 system the language, and show that the system speaks it better than the same recipe trained on the pool's existing transcripts.

Choose your language and your data

Pick one of the four languages. Your raw data pool is that language's subset of Vaani (IISc and ARTPARK): config *audio/<Language>* of the Hugging Face dataset *ARTPARK-IISc/Vaani* at revision 9e6ed4f94f7060e633cbc500480623ae7521522c. It is free under CC-BY-4.0; request access on Hugging Face.

Language	Hours of audio	Hours transcribed
Bhojpuri	490	27
Chhattisgarhi	407	18
Magahi	111	6.5
Maithili	466	21

The pool is spontaneous, image-prompted speech from many speakers and districts, and only a small share is transcribed. Clips are 16 kHz mono WAV, mostly 2–11 seconds long, and each language's config is 12–56 GB. Many clips have background noise, and many clips labelled Magahi or Maithili are closer to Hindi, so selecting usable speech in your language is part of the task.

What you can and can't add

Allowed	Public pretrained checkpoints released before this PS, even if they include the language. Text from any source. Speech in other languages.
Not allowed	Other recordings or synthetic speech in your Part 1 language, including from APIs. For that language, the pool is your only source of speech.

Name your language on Discord within one week after the release of this problem statement. Each language needs at least two teams, because held-out sentences come from other teams. If only one team chooses a language, a judge asks that team on Discord to choose a different language.

Directions you might take:

- **Selecting data:** language identification, and speaker and quality filtering, to find the clips worth transcribing.
- **Cleaning and segmenting:** denoising, removing music, diarization (who spoke when), and splitting into utterances.
- **Getting transcripts:** training ASR on the pool's transcribed share, or bootstrapping from a related language's ASR, with confidence filtering.
- **Scoring each pair automatically, so the worst data is filtered out.**
- **Training-side work:** transfer from a related language, or weakly supervised training.

The bar is intelligible speech, not studio quality: something instead of nothing.

Optional extras:

- Keep the pool's spontaneous, conversational style in the trained voice, instead of filtering it down to read-like speech. Most Indian training data is narration.
- Run the same pipeline on a second underrepresented language, to show it generalizes.

The test set you submit at the mid-term

Write 100 new sentences in your language, in Devanagari, of 5–20 words each, on everyday topics. Put them in *inputs/sentences.csv* with columns *sentence_id* and *text*.

- Don't take them from Vaani or any public text, and don't share them with other teams.

- The organizers release them to the other teams in your language when they release the evaluation inputs. No team is evaluated on sentences it wrote.
- Before teams generate outputs, the organizers' native listeners can reject a selected sentence. They can reject it only if it isn't an understandable sentence in your language. The organizers record the reason and take the next sentence in hash order. If fewer than 30 sentences remain, use all of them.

The dataset freeze

By the freeze, lock two complete manifests. *pipeline_manifest.csv* is your pipeline's final output, which is exactly the Part 1 data your final system trained on. *baseline_pipeline_manifest.csv is the baseline. Neither may repeat audio under* different IDs.

Build the baseline manifest like this:

1. Start with every pool clip that has a non-empty transcript.
2. Drop clips whose transcript contains [unintelligible], [inaudible] or <UNKNOWN_SEGMENT>, in any letter case.
3. In the rest, delete every other tag in angle or square brackets, such as <pause>, <static_noise>, </static_noise> or [breathing], keeping the words between paired tags.
4. Delete any text in curly braces. It is Vaani's standard or English spelling of the word just said.
5. Delete --, then collapse whitespace and drop transcripts left empty. Make no other change.
6. Keep each remaining clip once, with its original audio.

Each manifest includes stable *pair_id* values, transcripts, audio paths and audio SHA-256 hashes, with rows sorted by *pair_id*. Record both manifests' SHA-256 hashes in *freeze.txt* and include the unchanged manifests in *code.zip*.

After the random seed is released, use the selection procedure in Appendix A to sample 200 pairs from each manifest. If a manifest has fewer than 200 pairs, use all of them. Put both samples in *outputs.zip*, labelled by pipeline for scoring, alongside the held-out sentences read by your system and by the voice-gain baseline. This fixes both sampling populations before the seed is known.

How Part 1 is evaluated (5 results points)

Pipeline precision (2.5 points). The organizers check 400 pairs: the 200 sampled from your pipeline and the 200 from the baseline manifest. The pairs are mixed and hidden, so listeners don't know which pipeline produced which. At least two of the organizers' native listeners check each pair once. A listener marks a pair correct only if all three of these hold:

- the text is exactly what is said;
- the audio is clean;
- the speech is in your Part 1 language rather than Hindi. Hindi or English words mixed into a sentence are fine.

A pair's score is the share of its listeners who marked it correct. Precision is the average pair score, computed separately for your pipeline (x) and the baseline (b). The ceiling is 100%. These are separate output samples, so organisers use the independent-sample bootstrap specified in Scoring. The checks can start as soon as the seed is released.

Voice gain (2.5 points). The seed picks 30 held-out sentences written by other teams in your language, and one speaker from the pooled Part 2 enrollment clips. Your system and the voice-gain baseline both read those sentences in that speaker's voice. The baseline differs from your system only in its Part 1 training data. It is trained on the baseline manifest instead of your pipeline's output, with the same starting checkpoint, other data, seeds, hyperparameters, training steps, checkpoint selection, inference settings and any later training.

At least 10 of the organizers' native listeners first hear the enrollment clip, then compare the two versions of each sentence once, blind, and pick the better one. A version loses if the listener can't understand it or it doesn't sound like the enrolled speaker. If both versions fail, the comparison counts as a loss for your system. A sentence's score is the share of its listeners who preferred your system, and the measure is the average over sentences. 50% means no gain, and the ceiling is 100%.

PART 2: MY VOICE, MY LANGUAGES

The problem. Many Indians speak three or four languages. Clone someone's voice from a short clip today, and the clone either speaks only the recorded language or loses the person's voice when it switches. Large models are starting to do this well. Doing it well for Indian languages, at a size a team can train, is still largely open.

Your task. Given at most 30 seconds of a person speaking one language (Hindi, Tamil or English), synthesize them speaking Hindi, Tamil, Indian English and your Part 1 language. Further languages are optional. The output must sound like the same person and be clearly intelligible in each language.

Pronunciation should be native-like in each language. It is judged against how a native speaker of the target language says it, not against the enrolled speaker's own accent. Keeping the person's own accent counts under novelty, if you show it against real reference speech. Your Part 1 language is measured through Part 1's voice-gain test, which uses a cloned voice.

Two routes you can take:

- **Architecture:** change the model itself, for example by separating "who is speaking" from "which language's sounds" inside the model, or with new speaker or language conditioning.
- **Post-processing:** generate with a strong native-speaker voice, then convert it to the target speaker's voice (voice conversion), or refine the output after generation.

You can combine the two routes. Explain why your route works.

Optional extras:

- **Controllable delivery:** speak the same words at a requested pace or tone (calm, warm, urgent) while keeping them intelligible and keeping the speaker's identity. Test whether listeners recognize the intended delivery.
- **Expressive prosody:** questions marked only by pitch (aap kal aa rahe ho?), particles like *na* and *ji*, natural fillers like *haan ji* and *accha*.
- **Conversational from narration:** making the voice sound conversational when most of its training data is read narration.

The test set you contribute at the mid-term

Record three consenting multilingual speakers who aren't in any public dataset. Use Hindi for one speaker's enrollment clip, Tamil for another's and English for the third's. For each speaker, provide:

- an enrollment clip of at most 30 seconds in one language;
- 10 reference sentences they recorded in each of the other two languages;
- signed consent on the template in Appendix D. Your team keeps the signed forms and shows them to the judges on request; don't put them in *test_set.zip*.

Recording specification

Item	Requirement
Speaker	At least 18 years old
Room	Quiet, with no music or other voices; one speaker per clip
Audio format	Mono WAV at 16 kHz or higher, with at least 16-bit samples
Enrollment clip	10–30 seconds of natural speech
Reference sentences	New sentences of 5–20 words on everyday topics. Not taken from public datasets, and no personal information
Folders	Enrollment clips and sentence text go in *inputs/*. Reference recordings go in *answers/*

How Part 2 is evaluated (5 results points)

You synthesize every reference sentence for every pooled speaker except your own. An item is one reference sentence for one pooled speaker, and its target language is Hindi, Tamil or English. Compute each measure separately for each target language, then average the three languages with equal weight.

Measure	Points	How it is measured
Speaker similarity	2	Cosine similarity between the enrollment clip and your output, using microsoft/wavlm-base-plus-sv, averaged over items. The ceiling is the same average for the speaker's real reference recordings.
Intelligibility	1.5	Character error rate from openai/whisper-large-v3, averaged over items. The ceiling is the same average for the real reference recordings.
Naturalness	0.75	Listener ratings, 1–5, as described below. Naturalness is how much the clip sounds like a real person speaking.
Pronunciation	0.75	Listener ratings, 1–5, as described below. Pronunciation is how close the clip is to a native speaker of that language.

For naturalness and pronunciation, the seed picks 25 items per language. At least 10 of the organizers' native listeners hear each item three ways (your output, the baseline's output and the real recording), blind and mixed together. Each listener rates each clip once, with separate 1–5 ratings for naturalness and pronunciation. For each measure, average the listeners' ratings for each item, then average over items. The ceiling is the same score for the real recordings.

Consent. Use public datasets whose licences permit research use. Any new recordings need the speaker's written consent, including consent to clone their voice. Don't publish a cloned voice of anyone without that consent.

PART 3: SAY IT RIGHT

The problem. Voice agents in India read text written by LLMs, and that text is messy:

- **Hinglish typed in English letters, where some words look English but are** Hindi. In “woh log kal yahan the”, *the* is थे (“were”), not English “the”.
- **English words with Tamil endings, like meeting-ல (“in the meeting”).**
- **Amounts, dates and IDs: ₹2.5 cr, 4:30, a PNR, a UPI ID, a vehicle number.**
- **Indian names, often romanized in ways that don’t show how they are said.**

TTS systems misread these all the time, and one misread amount or name can break a call. A sentence that switches language is also a small cross-lingual cloning problem: the voice has to stay the same person across the switch.

Your task. Make your system read this kind of text correctly and naturally, in Hindi–English and Tamil–English, in a voice cloned with Part 2. Your contribution must be **learned**, and it is scored against a prompted LLM, not against hand-written rules. Learned means your contribution includes model weights your team trains or fine-tunes on data. Prompting, hand-written rules or lookup tables alone don’t count as the contribution, though they can sit beside a trained component when your controlled comparison shows what the trained component adds.

The scored baseline

Item	Detail
What it is	Your Part 2 voice system without your reading contribution, reading text that an open LLM has rewritten using the prompt and settings in Appendix C, unchanged.
Voice system	Your final system’s checkpoint from before your Part 3 training, with its own hash in *freeze.txt*. If your Part 3 contribution doesn’t change the voice system’s weights, it is the same checkpoint as your final system.
LLM	sarvamai/sarvam-m (24B parameters, Apache-2.0) at revision 01534a53c46f2788e392dbb3d994e0fa8f04d3fd, run with 4-bit or higher precision.

Item	Detail
Your training data	Any LLM that generates your Part 3 training data must have at most 24 billion parameters. Freeze its checkpoint, prompt and inference settings. Any model-generated training data for this part must come from open models.
Your own rules	You may report your own normalization rules as an additional comparison.

Two routes you can take:

- **Learned frontend:** train a model that rewrites raw agent text into something your voice system reads correctly. That means the right language and script for each word, numbers and IDs expanded the way a bilingual speaker would say them, and names respelled or converted to phonemes. A design that works on streaming text with only a few words of lookahead counts under novelty.
- **End to end:** fine-tune the speech model in your voice system to read raw mixed text directly, with no separate frontend.

Efficiency can count under novelty and depth. Compare your complete system with the complete baseline (prompted LLM plus voice system) on the same hardware and inputs, and report model sizes, inference settings and time to first audio. Speed only counts if your results aren't clearly worse than the baseline's.

Optional extras:

- **Phoneme-level pronunciation control:** a shared sound inventory for Indian languages, so a user can fix a mispronounced name by its sounds.
- **Fix one word:** repair a mispronounced name or brand term inside an existing clean recording without re-recording it. Regenerate only that word with the right pronunciation, so it fits the surrounding voice and recording. Measure whether the word is now right and whether blind listeners can spot the edit.
- **Written to spoken:** turn formal written language into natural spoken language, for example literary Tamil into spoken Tamil, preserving meaning, names, numbers and the intended politeness. Compare speech from the converted text with speech from the original text, using the same voice.

Starting data. Transliteration sets (Dakshina, Aksharantar), code-mixing sets (LinCE, GLUECoS), and synthetic agent replies you generate. Record generated data in your data manifest.

The test set you contribute at the mid-term

Write 50 realistic agent replies of at most 40 words in each of Hindi–English and Tamil–English, from domains such as banking, healthcare, travel and government services.

- Write Hindi–English replies in Latin script, like the first example above, and Tamil–English replies in Tamil script with English words in Latin script, like the second.
- Every reply contains at least one amount, date, time, ID or name.
- For every amount, date, time and ID, list all accepted readings under the policy in Appendix B. *dhai crore* and "two point five crore" are both right for ₹2.5 cr.
- For every name, include a reference recording of the accepted pronunciation.
- Also record each reply read naturally by a native bilingual speaker, with signed consent on the Appendix D template.
- The replies go in *inputs/*. The accepted readings, name recordings and reply recordings go in *answers/*.

How Part 3 is evaluated (5 results points)

The seed picks 25 pooled replies per language pair, none of them your own, and one pooled Part 2 speaker. You synthesize those 50 replies in that speaker's voice, and the baseline does the same. At least five of the organizers' native listeners hear each reply three ways (your output, the baseline's output and the real recording), once each, blind and mixed together.

Facts and names (3 points). For each amount, date, time and ID, the listener writes down the words they heard, in any script, without converting them to digits or symbols. A written answer is correct when it has the same words in the same order as an accepted reading, ignoring script, spelling, spacing and punctuation. A reading the contributor didn't list is also correct if it follows Appendix B; the organizers mark it as unlisted. For each name, the listener plays the name's reference recording and answers whether the reply said the name the same way. The organizers record each match decision.

A listener's score for a reply is the share of its facts and names they got right. A reply's score is the average over its listeners, and the measure is the average over replies. The ceiling is the same score for the real recordings.

Mixed-language reading (2 points). For each reply, a listener answers three yes-or-no questions and passes the reply only if all three answers are yes. A reply's score is the share of its listeners who passed it, and the measure is the average over replies. The ceiling is the same score for the real recordings.

- **Right language**: was every word said in its own language's sounds? Answer no if any word is read in the wrong language, such as the Hindi *the* read as English "the", or if an English word isn't said the way an Indian English speaker says it.
- **Same person:** did the voice stay the same person across every switch? Answer no if the voice sounds like a different speaker, or its quality jumps, at any switch.
- **Natural:** does it sound like a fluent bilingual speaker? Answer no for unnatural pauses at switches, misplaced stress or a flat, robotic reading.

PART 4: JUDGE WITHOUT A JURY

The problem. The standard way to check TTS quality is a MOS test, where people listen and rate. It is slow and expensive, so models get trained blind between rare listening tests. Automatic predictors exist, but they were built mostly on non-Indian speech. Speech-recognition round trips also hide the errors that matter in Indian languages: the recognizer's language model "fixes" a word that was mispronounced. For example, पल (*pal*, "moment") and फल (*phal*, "fruit") differ only by a puff of air.

Your task. Build an automatic evaluator for Indian TTS that agrees with native listeners on all three criteria, in Hindi and Tamil:

- **Naturalness:** does it sound human?
- **Pronunciation accuracy:** are the sounds right, including contrasts that change meaning, like aspiration (*pal*/*phal*), retroflex vs dental (ट/त), double consonants (*patā*/*pattā*), vowel length and nasal vowels?
- **Speaker similarity:** does it sound like the intended person?

Your metric can be language-specific, speaker-specific or language-agnostic. One natural direction is a minimal-pair test: a set of word pairs that differ by one sound, with a classifier that checks which word the TTS actually produced. Use your evaluator on your own Parts 2 and 3 during development. That use isn't scored, but it is a natural first test of the evaluator.

Optional extras:

- **Delivery:** judge whether speech carries a requested pace or tone, or the prosody a sentence needs, such as a question marked only by pitch.
- **Conversational naturalness:** judge whether speech sounds conversational rather than read.
- **Explained scores:** point to the word or sound that was mispronounced, not only a number.

Input and output contract

Your evaluator is a command-line tool. It reads a manifest CSV and writes a score CSV. The baseline must return the same fields.

Input column	Meaning
clip_id	ID of the clip
audio_path	Path to the audio
language	Hindi or Tamil
text	What the clip was meant to say
reference_audio_path	Reference audio, for speaker similarity
target_word, contrast	The two possible words, for minimal-pair items

Columns a criterion doesn't need may be empty. All required audio, including speaker-reference audio, belongs in *inputs/*.

Output column	Meaning
clip_id	One row per clip and criterion
criterion	naturalness, pronunciation or speaker_similarity
score	Your score
predicted_word	For each minimal-pair item: the word in target_word, the word in contrast, or unclear

The test set you contribute at the mid-term

Provide 100 to 150 clips of at most 20 seconds in each of Hindi and Tamil, in the input format, from at least three TTS systems plus human recordings. Your own Part 2 system may be one of the three. Include at least 20 minimal-pair items per language. Submit the clips with *answers/clip_metadata.csv*. Then the organizers' native listeners annotate every clip on all three criteria, with two listeners for each clip who work independently. The organizers add their answers to your set:

File	What it holds
answers/ratings.csv	clip_id, criterion, rater_id, rating. Each clip has two 1–5 ratings for each criterion. A naturalness rating can't stand in for pronunciation or speaker similarity.
answers/heard_words.csv	clip_id, rater_id, heard_word. For every minimal-pair item, each listener records the word they heard, choosing target_word, contrast or unclear in the same way as the output contract. Store the actual word, not the column name. Keep both listeners' original answers, including disagreements.
answers/clip_metadata.csv	clip_id, system_id, text_id. Identify the source system or human recording and assign the same text_id to clips with the same intended text in the same language. Keep this metadata out of the evaluator's inputs.

The set must include the same texts rendered by different systems, and clips compared for speaker similarity must share the same target-speaker reference.

How Part 4 is evaluated (5 results points)

Your frozen evaluator is scored on other teams' sets.

Measure	Points	How it is measured
Agreement with listeners	3	Spearman correlation between your scores and the listeners' average rating. Compute it within each qualifying contributed set for each criterion and language, because each set has its own listeners. Average over sets for each criterion and language, then over the six criterion–language combinations. Use clips where the two ratings for that criterion differ by at most one point. The ceiling is the correlation between those two listeners on the same clips, averaged the same way.
Word identification	1	Accuracy of predicted_word against the agreed heard_word on minimal-pair items. Include an item only when both listeners give the same word label, including unclear. The ceiling is 100%.
Pairwise ranking	1	For naturalness and speaker similarity, the accuracy of ordering pairs of clips of the same text from different systems, averaged over the two criteria. Within each contributed set, use every pair with matching text_id and language, different system_id values, and a strict preference in the same direction from both listeners for that criterion. Speaker-similarity pairs must share the target-speaker reference. Count each pair once. A predicted tie is incorrect. The ceiling is 100%.

Coverage and edge cases

Usable coverage. A contributed set counts toward agreement for a criterion and language only if, after the filters above, it has at least 30 usable clips, each listener's ratings vary across those clips, and the listeners' average ratings also vary. This makes both the agreement score and its human ceiling calculable. A set counts toward word identification with at least 10 agreed minimal-pair items in that language, and toward pairwise ranking with at least 10 usable pairs for that criterion and language. The organizers report the coverage of each set with the mid-term feedback. If your set falls short, fix it after the mid-term feedback. If you don't fix it, it is left out of the pool where it falls short.

Undefined scores. If an evaluator gives every clip in a set the same score for a criterion, its correlation on that set is 0. A missing score or predicted_word is a missing output (see Scoring), and an invalid predicted_word counts as wrong. These rules apply to the baseline too. In a bootstrap replicate, use a correlation of 0 whenever either vector being correlated is constant. Use the same sampled clips for the system and baseline, and keep the qualifying sets and averaging weights fixed across replicates.

Unavailable measures. A criterion and language with no qualifying set is left out of that measure's average. If word identification or pairwise ranking has no qualifying data at all, its points move to agreement with listeners. When agreement has no qualifying data, divide the full 5 points equally among the remaining available measures. If none of the three measures has qualifying data, award 2.5 of the 5 results points as a neutral score and report that Part 4 could not be measured. These fallbacks apply only to missing qualifying evaluation data; missing system outputs still follow the scoring rules.

TRACK 2: RECURSIVE SELF-IMPROVING SPEECH SYSTEM

THE PROBLEM

Most speech is unlabelled. It has no transcript and no description of how it is said, and voice models need both. A speech captioner can describe speech, a voice generator trained on those descriptions can produce speech for chosen targets, and that speech can in turn train the captioner. This track asks whether the loop really improves itself: better understanding makes better generation, which makes better understanding.

THE CHALLENGE

For Hindi and Tamil, build a captioner and a generator, and run the loop for at least two rounds after round 0. In round *r*:

1. Captioner C(r−1) describes the unlabelled pool, creating a training set.
2. Generator G(r) is trained on that set to produce speech from text and a description.
3. G(r) simulates speech that targets C(r−1)'s weak areas, and that speech trains captioner C(r).

C(r) relabels the pool at the start of the next round. Your final system is the captioner and generator from your last completed round, at least round 2.

Descriptions. The captioner describes everything in a clip beyond the words, in the fixed format in Appendix E: arousal, valence and dominance (how energetic, how positive and how in control the voice sounds, each 1–5), emotion, timbre, timed non-speech events such as laughs, sighs, breaths, coughs and fillers, and a free-text description of the voice and delivery, the way a voice coach would write it. Pitch, pace and loudness aren't described. They are measured with the fixed tools in Appendix E. Transcripts can come from any public ASR model. Generated speech has known targets but not known contents, so treat its targets as noisy labels.

The unlabelled pool. Vaani audio/Hindi and audio/Tamil at revision 9e6ed4f94f7060e633cbc500480623ae7521522c: about 14,900 and 850 hours (2.8 TB and 177 GB). Use any subset, plus any other public speech without labels. Treat the pool as unlabelled. Vaani's transcripts and metadata can help you find files and split data, but they can't serve as training labels.

Round 0: your scored baseline

Item	Detail
Captioner	Qwen/Qwen3-Omni-30B-A3B-Captioner at revision a2bd106cbf527db5676e79662674da22b0545ec0, run with its own audio-only interface.
Generator	ai4bharat/indic-parler-tts at revision 660bc53c64ae83150613264111708a0465211237, which already speaks from text descriptions.
Conversions	A frozen parser or open text model converts the captioner's English caption to the Appendix E format: use 3 for a missing rating, neutral for a missing emotion and empty lists for missing events or timbre, and don't invent event times. A frozen conversion turns Appendix E targets into the generator's description prompt.
Freezing	Freeze both conversions before round 1 and use them unchanged in every round, with the same decoding settings and precision for round 0 and your final system wherever the interfaces allow.
Submission	Submit round 0 in *round0.zip* at the mid-term. It can't change afterwards, which overrides the shared rule on replacing baselines.

Model size. Later rounds may train adapters or smaller student models, keeping the 30B captioner as a frozen teacher. Start with a few hours per language and measure the cost of one round before you scale up.

Where training labels come from

After round 0, every training label and description must come from your own loop models, public ASR transcripts, the Appendix E tools, or deterministic augmentation (its applied settings, and inserted event types and times). Every later checkpoint must descend from recorded loop updates.

- Don't train on human-labelled emotion or speech-description data at any stage.

- Other captioners may audit results but can't supply labels, and commercial APIs are for additional comparisons only.
- People may remove clips only for corruption, unreadable audio, personal information or missing permission, with the reason logged. Any other selection must use automatic rules recorded before the round.
- These rules override the shared rules on models and APIs.
- Keep a per-example manifest in *code.zip* (source ID, audio hash, round, label producer and its checkpoint hash, settings, and any removal reason), and record each round's manifest hash in *freeze.txt*.

Directions you might take:

- **Targeting weak areas:** find where the captioner is least accurate, and simulate speech that varies only those attributes.
- **Ground truth from outside the loop:** signal processing for attributes it can set exactly, such as noise, reverberation, pitch and tempo, and real non-speech sounds spliced in at known times.
- **Guarding against a private code:** auditing each round with a second captioner from a different model family, so the generator doesn't learn cues only its own captioner hears.
- **Confidence filtering and self-consistency when relabelling the pool.**

The test set you contribute at the mid-term

For each of Hindi and Tamil, record 25 groups from at least five consenting native speakers. A group is one sentence recorded four times by the same speaker, with delivery that differs in emotion, energy, voice quality or non-speech events, not only in pitch, pace or loudness.

- Each clip is at most 20 seconds.
- Mark 12 groups *describe* and 13 *simulate*, with no speaker or sentence in both.
- Then the organizers' native listeners label every clip in the Appendix E format. Two listeners label each clip independently, and each listener writes a free-text description in English.
- Recordings follow Part 2's recording specification, and consent and signed forms are handled as in Part 2. The file layout is in Appendix E.

- If the mid-term feedback finds problems, repair your set by the date announced on Discord.

How Track 2 is evaluated (20 results points)

The seed picks the sets of up to three other Track 2 teams (Appendix E). Your final system, round 0 and your control (see Showing the loop) run on all their items. Scoring uses the shared levels, confidence intervals and gap closed.

Group	Measure	Points	How it is measured
Describe (7 points)	Arousal, valence and dominance	2	Spearman correlation between the captioner's ratings and the listeners' average, computed within each set for each dimension and language, then averaged. Use clips where the two listeners' ratings differ by at most one point. The ceiling is the correlation between the two listeners. Part 4's usable-coverage, undefined-score and unavailable-measure rules apply.
	Emotion, events and timbre	2	The average of three values: emotion accuracy on clips where both listeners chose the same emotion, event F1, and timbre F1 on tags both listeners chose, with matching as defined in Appendix E. Each ceiling is 100%. A component with no reference labels is left out.
	Free-text description	3	The seed picks 20 describe groups per language and one target clip in each. A listener reads a description of the target, hears the group's four clips in hash order, and picks the one described. Five listeners judge each group with the description from your captioner, and five other listeners judge it with the description from round 0. No listener sees a group twice. The measure is accuracy; chance is 25%, and the ceiling is 100%.

Group	Measure	Points	How it is measured
Simulate (7 points)	Measurable control	2	The generator renders every simulate input, and the Appendix E tools measure each output. The measure is the share of pitch, rate and loudness values within tolerance of their targets, averaged per item, then over items and languages. The ceiling is 100%.
	Description match	4	The seed picks 20 simulate inputs per language. A listener sees the transcript and the full target description, hears your output and the output of round 0 blind, and picks the one that better matches the requested delivery. If the listener can understand only one output, that output wins. If the listener can understand neither, it counts as a loss for your system. Five listeners judge each input. The measure is the share of judgments that prefer your system, averaged over listeners, inputs and languages. 50% means no gain, and the ceiling is 100%.
	Intelligibility	1	Character error rate of your outputs, computed with Whisper and the normalization of Part 2, with Hindi and Tamil weighted equally. The ceiling is 0% error.
Loop gain (6 points)	The four automatic measures above	1.5 each	Arousal, valence and dominance; emotion, events and timbre; measurable control; and intelligibility are scored again, with your control as the baseline instead of round 0. Loop gain rewards gains that come from the loop itself, not only from training on your own data. If the control is missing outputs, loop gain scores level 0.

The organizers' native listeners judge these measures, in sessions of at most 40 clips. If either model or its round-0 baseline is missing outputs, every measure for that model scores level 0.

Showing the loop

- Before round 1, freeze a real-audio development set with your own human labels. Never use it to choose training data, prompts or checkpoints. Report every round on it.
- Also run one control. The control is different from your loop only in the source of its labels and simulations: it uses the round-0 models for all of them. It uses the same architecture, code, starting checkpoints, data budget, training steps, seeds, checkpoint selection and inference settings. Freeze it with your final system, and run it on every evaluation item. The control is the baseline for loop gain. Without this control, Rigour is capped at 1.

Name the captioner or the generator as your lead part.

BOTH TRACKS: HOW EVALUATION WORKS

Evaluation sets. At the mid-term, every Track 1 team submits a test set for each part, and every Track 2 team submits its Track 2 test set. Test-set quality is scored at the mid-term (see Scoring). In Track 2, the captioner is evaluated on other teams' describe groups and the generator on their simulate inputs. In Track 1, the final evaluation items depend on the part:

Part	Final evaluation items
1	Other teams' held-out sentences in your Part 1 language, plus the frozen manifests of your pipeline and its baseline. The seed selects 30 sentences, one pooled Part 2 speaker to read them, and 200 pairs from each manifest, 400 pairs in total.
2	Other teams' speaker sets, covering Hindi, Tamil and English enrollment.
3	25 of other teams' replies per language pair, chosen by the seed, read in the voice of one pooled Part 2 speaker that the seed picks.
4	Other teams' labelled clips in Hindi and Tamil.

Different items for each team. Pooled items never include your own contributions, so each team is scored on a slightly different selection. The seed ranks the same candidates in the same order for every team, so in Parts 2–4 most selected items are shared. Your system and its baseline always use exactly the same items.

Inputs and answers. Each test set has two folders. inputs/ holds what a system is given. *answers/* holds the scoring references and labels. Nothing in *answers/* may be needed to produce an output. In Part 2, target-sentence recordings are answers. Part 3's accepted readings, name recordings and reply recordings are answers. In Part 4, speaker-reference audio is an input, while criterion ratings, heard-word labels and clip-grouping metadata are answers.

The final window, step by step

1. **Freeze .** Submit freeze.txt with your code commit hash, the SHA-256 hash of every model checkpoint, and the exact generation command and its seed, for your system and every scored baseline. That includes Part 1's precision-baseline pipeline and voice-gain baseline, and the hashes of both complete Part 1 pipeline manifests.
2. **Inputs released .** The organizers release the pooled inputs for every Track 1 part and for Track 2, including Part 1's held-out sentences, and a random seed. The organizers' Part 1 precision checks use the seed with your two frozen manifests, and the checks can start.
3. **Outputs locked .** Run your frozen system and every scored baseline on the evaluation items for each part. Put all outputs in *outputs.zip*, plus Part 1's sampled pairs, then submit *outputs.zip* and *outputs_hash.txt* with the ZIP's SHA-256 hash. Generate one output per input. Don't edit outputs or pick between several generations.
4. **Answers released .** The organizers release the scoring answers for Track 1's Parts 2–4 and for Track 2. Score your locked outputs on the automatic measures. The organizers run all the listening tests, including the Part 1 checks of the sampled pairs and the synthesized sentences.
5. **End term submission . Your *outputs.zip* must match the hash** you submitted.

Sampling and confidence intervals. The random seed determines the evaluation samples, speakers and listening order. Follow the selection and bootstrap procedures in Appendix A. The appendix is required and forms part of the frozen evaluation rules. The organizers compute every listener-based measure and its confidence interval with these procedures. Teams compute the automatic measures.

Listening tests

- The organizers recruit and manage all listeners. Teams don't recruit listeners or run listening tests. Listeners are native speakers of the language, and they are not members of any team.
- Listening is blind: listeners don't know which system produced a clip, and the order is randomized.
- Each listener rates each selected clip once.

- The organizers run the Part 1 precision checks after the release of the random seed. They run all other listening after the outputs are locked, in Hindi, Tamil, English and the Part 1 languages. They share the listening results of each team after the evaluation window.

Fixed measurement tools

Use the named tools: speaker similarity with microsoft/wavlm-base-plus-sv, and character error rate with openai/whisper-large-v3. Run Whisper with the language set to the sentence's language. Before computing character error rate, apply Unicode NFC normalization to both texts, lowercase them, remove punctuation and collapse whitespace. You can also use your own extra metrics next to these tools.

Verification

- **Automatic measures.** Judges re-run the scoring of the top-ranked teams from their frozen code. A re-run within 2% of the reported value, relative, counts as a match.
- **Review.** If a check is flagged, you get 24 hours to explain. If the difference is an honest error, the values of the judges replace yours, with no penalty.
- **Violations.** Fabricated data, edited outputs or outputs that don't match the hash cost all results points, and the case is reported to the organizing committee.
- **Close calls.** If teams near a top-three boundary are within 2 total points of each other, the organizers collect more listening ratings for those teams.

Scored baselines

Each part has its own scored baseline. Use reproducible baselines with publicly available code and model weights that you can run and freeze. Choose the strongest eligible baseline that fits each part's specified comparison, and get your baselines approved by a judge on Discord within one week after the release of this problem statement:

Part	Scored baseline
Part 1	Fixed, as defined in Part 1: the baseline manifest for precision, and the same TTS training recipe trained on that manifest for voice gain.
Part 2	A public cross-lingual voice-cloning system. To be eligible, it must have public code and weights released before the PS, clone a voice from 30 seconds of audio or less, and support Hindi, Tamil and English. If no eligible system supports all three, judges approve the closest one.
Part 3	The prompted LLM defined in Part 3.
Part 4	An existing public evaluator for each criterion.
Track 2	Round 0, as defined in Track 2, and your control for loop gain.

The required controlled comparison may be an additional experiment if it differs from the scored baseline.

Additional comparisons. You may report commercial API results separately, naming the provider, model/version, request settings and run date. They don't set the baseline value used for results points and are excluded from the checkpoint-freeze requirement. Keep their returned outputs with the comparison evidence in *evaluation.zip*.

SCORING

Every team gets a score with the same rubric, and all teams from both tracks are ranked together. The results measures are different for each track: Track 1's are below, and Track 2's are in its evaluation section.

Stage	Criterion	Points
Mid-term (30)	Problem framing and prior work	10
	Baselines and evaluation setup	5
	Contributed test sets	5
	Early results and revised plan	10
Final (60)	Results (computed)	20
	Novelty and depth: lead part	14
	Novelty and depth: rest of the system	6
	Rigour	10
	Reproducibility	10
Presentation (10)	Final Presentation	8
	summary.md	2

Results are computed from your verified measurements. In Track 1, each part is worth 5 results points, divided among its measures:

Part	Measure	Points	Better	Ceiling (c)
1	Pipeline precision	2.5	Higher	100%
	Voice gain	2.5	Higher	100% (b is fixed at 50%)
2	Speaker similarity	2	Higher	Real reference recordings
	Intelligibility (character error rate)	1.5	Lower	Real reference recordings
	Naturalness	0.75	Higher	Real reference recordings
	Pronunciation	0.75	Higher	Real reference recordings
3	Facts and names	3	Higher	Real reply recordings
	Mixed-language reading	2	Higher	Real reply recordings
4	Agreement with listeners	3	Higher	Agreement between the two listeners
	Word identification	1	Higher	100%
	Pairwise ranking	1	Higher	100%

How each measure becomes points

Each part defines exactly how its measures are computed. For each measure:

1. Compute your value x and the scored baseline's value b on the evaluation items specified for that part. Compare both systems on the same items, except for Part 1 pipeline precision, which uses separate 200-pair samples from the two pipelines.
2. Compute the 95% confidence interval of $x - b$ with the bootstrap defined in Appendix A. It is paired for measurements on the same test items. For Part 1 pipeline precision, it resamples each pipeline's 200 judged pairs independently, because the two samples are separate. A common seed makes the calculation reproducible. It doesn't pair the two samples.

3. Compute the share of the gap closed, $g = (x - b) \div (c - b)$. The formula works the same way whether higher is better (similarity) or lower is better (error rate).
4. Convert the result to a level:

Level	Meaning
0	Missing outputs, or the whole confidence interval is on the worse side of the baseline
1	The confidence interval includes zero: no clear difference from the baseline
2	Clearly better, and *g* is below 0.25
3	Clearly better, and *g* is from 0.25 up to 0.5
4	Clearly better, and *g* is 0.5 or more

Each measure earns its points × level ÷ 4. Your results score is the sum of the results measures of your track, out of 20.

Two special cases:

- **Above the ceiling.** If you are clearly better than the baseline and your value passes the ceiling, *g* is above 1, which is level 4. This can happen where real recordings are an imperfect reference, such as speaker similarity across languages.
- **Ceiling no better than the baseline.** If c equals b, or the baseline is already on the better side of the ceiling, *g* can't be computed. The measure then scores level 2 when you are not clearly worse than the baseline, and level 0 when you are.

Every part must work. In Track 1, a part works when your frozen system and its scored baseline produce an output for every evaluation item in that part. If either is missing outputs, every measure in that part scores level 0.

Worked example (illustrative numbers)

Part 2 intelligibility is worth 1.5 points, and a lower error rate is better. Averaged over the three languages, your system's character error rate is x = 10.5%, the baseline's is b = 14.0%, and the real recordings' is c = 5.0%.

1. The 95% confidence interval of $x - b$ runs from −5.1 to −1.9 percentage points. All of it is on the better side of zero, so your system is clearly better.
2. $g = (10.5 - 14.0) \div (5.0 - 14.0) = -3.5 \div -9.0 = 0.39$.
3. Clearly better, with g from 0.25 up to 0.5, is level 3.
4. The measure earns 1.5 × 3 ÷ 4 = 1.125 points.

Everything else is judged

Each judge scores each criterion 0–4 against the anchors below; 1 and 3 fall between the anchors. Points equal the average judge score ÷ 4 × the criterion's weight. The total is rounded to two decimal places.

Criterion	0	2	4
Problem framing and prior work	Restates the problem statement	Relevant prior work, contribution stated	Precise gap in prior work, a testable hypothesis, and a reason it should work
Baselines and evaluation setup	No running baseline	Baselines run for every part	Plus controlled comparisons designed and errors broken down by category
Contributed test sets	A test set missing	Every required test set meets the format	Plus test sets that cover hard cases and, in Track 1, a Part 4 set that meets usable coverage
Early results and revised plan	No results	First results against the baselines	Results with failure analysis, and a revised plan that follows from it
Novelty and depth: lead part	Existing models used unchanged	A meaningful adaptation of known methods	A new method or insight explored in depth. A clearly explained negative result can reach 4
Novelty and depth: rest of the system	Existing models used unchanged throughout	A meaningful adaptation in at least one other part or model	Meaningful adaptations across the rest, all working in one system
Rigour	No controlled comparison for the lead part (Rigour is then capped at 1)	Controlled comparisons and some ablation	Controlled comparisons for every part, ablations, errors by category, confidence intervals, honest limits
Reproducibility	Doesn't run	Runs after fixes	Runs as documented, and the frozen system reproduces the submitted outputs

Criterion	0	2	4
Presentation	Unclear	Clear account of the work	Clear, shows deep understanding, and answers questions well
summary.md	Missing, or written for specialists	Clear account of the problem and the result	A non-specialist could explain what you built and found, with one concrete example

DELIVERABLES

Mid-term :

The mid-term checks that the direction, baselines and evaluation of each team are sound while there is still time to change course. It also collects the test sets that every team is evaluated on.

- *report.pdf:* at most 6 pages, plus references. Why: it carries the 30 mid-term points (problem framing, baselines and evaluation setup, early results and revised plan). Judges and mentors give feedback on it.
- *results.csv:* baseline and current numbers for each part on your development data. In Track 2, give the numbers for each round so far. **Why:** it shows in one comparable format that every baseline runs, and where your first results stand against it.
- *test_set.zip:* one folder for each part, each split into inputs/ and answers/ where applicable: Part 1's 100 held-out sentences, Part 2's speaker set, Part 3's agent replies with their recordings, and Part 4's clips, which the organizers label. In Track 2, *describe/* and *simulate/* folders for each language, which the organizers label. **Why:** every team is evaluated on the sets of other teams, so the sets must arrive early enough for the organizers to label, check and pool them. Their quality is scored (5 points).
- *round0.zip (Track 2):* the code, conversion settings and checkpoint hashes of round 0. **Why:** it fixes the Track 2 baseline before any loop results exist, so nobody can make it weaker later.

Freeze :

- *freeze.txt: code commit hash, checkpoint SHA-256 hashes, generation* commands and seeds, for your system and every scored baseline. Also the SHA-256 hashes of both complete Part 1 pipeline manifests. In Track 2, the checkpoint hashes of every round, including round 0 and the control. **Why:** it proves that the evaluated system and baselines are the ones that were fixed before the evaluation inputs and the seed were released.

Outputs lock :

- *outputs_hash.txt: the SHA-256 hash of your outputs.zip.* ***Why:*** *it fixes your* outputs before the scoring answers are released.
- *outputs.zip: the file described under the end-term submission, uploaded at* the outputs lock. **Why:** the organizers need your outputs immediately to run the listening tests.

End-term submission :

The end-term collects everything that is needed to score, verify and reproduce your final system.

- *report.pdf:* at most 8 pages in double-column conference format, excluding references, plus an appendix of at most 4 pages. Put the lead contribution, the main results for every part (in Track 2, both models and the loop) and the essential controlled comparisons in the main report. Use the appendix for additional experiments and implementation details. **Why:** it is the main evidence for novelty and depth and for rigour (30 of the final-stage points).
- *code.zip:* source that matches the frozen commit, including your selection, bootstrap and scoring code. Include a *README.md* with setup steps, one command for each part that reproduces its headline result, and pinned dependencies (*requirements.txt* or equivalent). Put model weights behind a download link in the README, matching the frozen hashes. Include Part 1's complete, unchanged *pipeline_manifest.csv* and *baseline_pipeline_manifest.csv, and Part 3's frozen baseline prompt.* ***Why:*** judges run the scoring again and reproduce your outputs from it. It is scored under reproducibility (10 points).

- *outputs.zip:* the outputs of your system and of every scored baseline on the evaluation items of each part, matching *outputs_hash.txt*. Include the 200 sampled audio–transcript pairs from each Part 1 pipeline (400 pairs in total), with their pipeline identities kept for scoring. Include Part 4's score CSVs with the predicted-word fields. In Track 2, include the descriptions from your captioner for every describe clip and the audio from your generator for every simulate input, for your final round, round 0 and the control. **Why:** these are the outputs that get scored, and they must match the hash fixed at the outputs lock.
- *evaluation.zip:* your measurements, the evidence for your automatic measures, and any additional comparisons. **Why:** judges can check your automatic measurements and additional comparisons.
- *data_manifest.csv:* every data source, its licence and the hours used. Why: it shows that every source is licensed and that you followed the data rules of Part 1 and Track 2.
- *demo.mp4:* at most 4 minutes. In Track 2, show one clip that round 0 and your final captioner describe, one description that both generators render, and one failure that remains. In Track 1, show one cloned voice of a speaker who agreed to the demo. The voice speaks your Part 1 language and reads a mixed-language agent reply, each next to the baseline. Also show one failure that remains. **Why:** judges can hear what the system actually sounds like next to its baseline.
- *summary.md:* at most one page, in plain language for someone who isn't a speech researcher: the problem, what you built, what you found, and one example. It is scored as its own presentation criterion. **Why:** it shows that you can explain the work to a person outside the field.

No other files are accepted. Extra files incur the standard Inter IIT penalty. The technical constraints, evaluation scheme and deliverables above are frozen at release.

RULES AND FAIR PLAY

- **Models.** Use any open model whose licence allows research, and name the exact checkpoint. There is no limit on model size. Closed commercial APIs may be used for additional comparisons or to generate training data (except Part 3's, or speech in your Part 1 language), never inside the submitted system or its scored baselines.
- **Compute and data.** There is no cap, apart from the data rules of Part 1 and Track 2. Report the GPU type, total GPU-hours and the hours of audio you used. We give no credit for using more compute or data.
- **Controlled comparisons (required).** Show the effect of your contribution in each part alone. Use the same starting model and conditions, with your contribution removed or replaced by a simple alternative. Without a controlled comparison for your lead part, Rigour is capped at a judge score of 1, which is at most 2.5 of its 10 points.
- **Data provenance.** Submit a data manifest listing each source, its licence and the hours used. Don't redistribute audio or text unless its licence allows it. Don't record anyone covertly, and remove personal information.
- **Anonymity.** Submissions must not reveal your institute. Use your team ID everywhere. Publish public repositories only after results are announced.
- **No Rime product is required.**

Controlled comparison examples

In Track 1, compare like this. In Track 2, each round is compared with the previous one.

Part	Example comparison
1	Same training recipe on data before and after your filtering method
2	Same base system with and without your conditioning or conversion method
3	Same voice system with the prompted-LLM baseline against your learned approach
4	The same evaluator with your contribution removed or replaced by a simple alternative, on the same held-out audio

GLOSSARY

Term	Meaning
TTS	Text-to-speech: software that reads text aloud
ASR	Automatic speech recognition: speech to text
MOS	Mean opinion score: listeners rate quality from 1 to 5
Diarization	Working out who spoke when in a recording
Alignment	Matching each piece of text to where it is spoken in the audio
Voice conversion	Changing who a recording sounds like while keeping what was said
Enrollment clip	The short recording of a person that a cloning system copies the voice from
Speaker embedding	A vector summarizing what a voice sounds like; similar voices give similar vectors
Code-mixing	Mixing two languages in one sentence, like Hinglish or Tanglish
Transliteration	Writing a word in another script: mera ↔ मेरा
Normalization	Turning symbols and digits into words, like ₹2.5 cr into “dhai crore”
Minimal pair	Two words that differ by one sound and mean different things (पल / फल)
Spearman correlation	How well two rankings agree, from −1 to 1
Ablation	Removing one part of a system to measure what that part contributes
Confidence interval	The range a measured score would likely fall in if the test were repeated
Random seed	A published number that fixes a random selection, so anyone can reproduce it

APPENDIX A: REQUIRED SAMPLING AND BOOTSTRAP PROTOCOL

This appendix is required and forms part of the frozen evaluation rules. Track 2's selections and bootstrap keys are in Appendix E.

Random selection

The random seed fixes every random choice, and every team computes it the same way, with no random-number library:

1. For each candidate, compute the SHA-256 hash of the text *seed|selection|item_id. Here seed is the published nonnegative integer* written in decimal without leading zeros, *selection* is the name in the table below, and *item_id* is the candidate's ID. Pooled items are identified as *team_id/item_id. Hash UTF-8 bytes with literal | separators and no added* spaces, byte-order mark or trailing newline. These encoding rules also apply to the bootstrap.
2. Sort the candidates by hash, read as an unsigned hexadecimal integer, from lowest to highest. Break any hash tie by ID in ascending UTF-8 byte order. IDs are case-sensitive; team IDs and local item IDs use only ASCII letters, digits, periods, underscores and hyphens, and local IDs must be unique within their submitted collection.
3. Take the first *m*.

Selection	Candidates	m
part1_sentences	Every pooled held-out sentence in your Part 1 language except your own	30
part1_pipeline_pairs	Every pair in pipeline_manifest.csv	200
part1_baseline_pairs	Every pair in baseline_pipeline_manifest.csv	200
part1_speaker	Every pooled Part 2 speaker except your own	1
part2_items_hi, part2_items_ta, part2_items_en	Every pooled Part 2 item in that language except your own	25 each

Selection	Candidates	m
part3_speaker	Every pooled Part 2 speaker except your own	1
part3_replies_hi-en, part3_replies_ta-en	Every pooled reply in that language pair except your own	25 each

Bootstrap

Confidence intervals resample evaluation items, not listeners: sentences for voice gain, pairs for precision, items in Part 2, replies in Part 3, and clips, minimal-pair items or ranking pairs in Part 4. Use the following procedure:

1. Sort the eligible items by ID in ascending UTF-8 byte order, using *team_id/item_id for pooled items. A ranking pair's ID is its two pooled clip IDs,* sorted in that order and joined with ~. Number item positions from 0 to $n - 1$, where n is the number of eligible items.
2. Run replicates $r = 1$ through 1,000. In each replicate, make n draws with replacement, numbered $j = 0$ through $n - 1$. Hash the exact text *seed|bootstrap|measure|r|j. Write r and j in decimal without leading zeros.* Read the full SHA-256 digest as an unsigned hexadecimal integer h, and select the item at zero-based position h mod n.
3. Use the exact measure keys below. Angle brackets mark substitutions and are not included in the hash input. *language* is hi, ta or en in Part 2, and hi or ta in Part 4; Part 4's criterion is naturalness, pronunciation or speaker_similarity.

- Paired measures use identical draws for the system and baseline. Part 1 precision uses separate keys for its two pipelines.
- Part 2 resamples each language separately, then averages the three languages.
- Part 4 agreement resamples each contributed set, criterion and language separately, then repeats the published averaging procedure.
- Pairwise ranking resamples each criterion separately, then averages the two accuracies.

Measure	Exact bootstrap key
Part 1 precision, system / baseline	precision_pipeline / precision_baseline
Part 1 voice gain	voice_gain
Part 2 speaker similarity	part2_speaker_similarity/<language>
Part 2 intelligibility	part2_intelligibility/<language>
Part 2 naturalness / pronunciation	part2_naturalness/<language> / part2_pronunciation/<language>
Part 3 facts and names	facts_and_names
Part 3 mixed-language reading	mixed_language_reading
Part 4 agreement	part4_agreement/<team_id>/<language>/<criterion>
Part 4 word identification	part4_word_identification
Part 4 pairwise ranking	part4_pairwise_ranking/<criterion>

Recompute x and b from the resampled items in each replicate, except that voice gain's baseline value b is fixed at 0.5. Store $x - b$ and sort the 1,000 differences from lowest to highest. The confidence interval endpoints are the 25th and 975th values, at zero-based positions 24 and 974, with no interpolation. Apply Part 4's undefined-correlation rule within each replicate; do not drop or redraw a replicate.

Worked example. With seed 123, measure facts_and_names, replicate r = 1 and n = 10 sorted items, the first three draws are:

Draw j	Exact UTF-8 hash input	Zero-based item position
0	123|bootstrap|facts_and_names|1|0	0
1	123|bootstrap|facts_and_names|1|1	8
2	123|bootstrap|facts_and_names|1|2	1

APPENDIX B: ACCEPTED READINGS POLICY

This appendix is part of the frozen evaluation rules. List every reading a fluent bilingual speaker would naturally say on a voice-agent call. A reading may use either language of the pair, or mix them the way a speaker would.

Type	Accepted	Not accepted
Amounts	Indian number words (lakh, crore; Tamil latcham, kodi) in either language; fractions such as dhai, saade or Tamil irandarai (“two and a half”) where a speaker would use them; decimal readings such as “two point five crore”; and the currency word before or after the number.	Reading a symbol literally (“rupee sign”), or converting to a different scale (“twenty-five million”) when the text uses lakh or crore.
Dates and times	Day and month in either language, with the day as an ordinal or a cardinal number; clock times as a speaker says them, such as saade chaar baje, Tamil naalarai mani or “four thirty”.	Reading the punctuation, as in “four colon thirty”.
IDs (PNR, UPI ID, vehicle number, phone number)	Letters as English letter names; a part of a UPI ID or email that is a word or name, such as rahul or sharma, may be said as that word; digits one at a time or in natural groups of two, in either language; “@” as “at” or “at the rate”; and “.” as “dot”. Every character must be read, in order.	–
Names	The pronunciation in the name’s reference recording. Spelling variants don’t matter, because listeners compare the sound with that recording.	–

APPENDIX C: PART 3
BASELINE PROMPT

This appendix is part of the frozen evaluation rules. Freeze the model and tokenizer revisions and the chat template. Send the prompt below as the only user message, using the model's chat template unchanged, without adding a system message, and with thinking mode turned off, replacing {reply} with the raw reply text. Decode greedily (temperature 0, one beam, no repetition penalty) with at most 1,024 new tokens. Pass only the newly generated text, with special tokens removed, to the baseline voice system unchanged. The prompt is the same for both language pairs.

Prompt

Rewrite the agent reply below so that a text-to-speech system can read it aloud correctly.
Rules:
1. Keep the meaning and the word order. Don't add, remove or translate anything.
2. Write Hindi words in Devanagari and Tamil words in Tamil script. English words may stay in Latin letters or be written as they sound in the reply's Indian script. A word may keep an English stem with a Tamil suffix, as in meeting- .
3. Write every amount, date, time and other number as the words a fluent bilingual speaker would say on a phone call, using Indian conventions such as lakh and crore.
4. Read IDs character by character: letters as English letter names, digits as number words, "@" as "at the rate" and "." as "dot".
5. Keep names as they are said, written in the script of the language they come from.
6. Output only the rewritten reply.
Reply:
{reply}

APPENDIX D: SPEAKER CONSENT TEMPLATE

Use this text, filled in, for every speaker your team records for a test set in Track 1's Parts 2, 3 and 4 or in Track 2. Each speaker signs before recording.

Recording consent: Inter IIT Tech Meet 15.0, Rime problem statement

- **Who is recording:** team [team ID], contact [name and email].
- **What is recorded:** [for example, a 30-second clip of me speaking Hindi, and 10 sentences in each of Tamil and English].
- **How it's used:** teams in this event, its judges and the organizers' listeners can use my recordings, and generate speech in my voice, only to evaluate systems for this event.
- **Who receives it:** the organizers share my recordings with the other participating teams and the judges, and the organizers play them to their listeners. Nobody will publish my recordings, and nobody will publish speech generated in my voice or use it outside this event.
- **Deletion:** all copies of my recordings, all speech generated in my voice, and voice data derived from them, such as speaker embeddings, are deleted by [deletion date announced by the organizers].
- **Withdrawal:** I can withdraw if I tell the contact above before the mid-term submission opens. My recordings are then not submitted.
- **Age:** I'm 18 or older.

Name, signature and date.

APPENDIX E: TRACK 2 DESCRIPTION FORMAT AND PROTOCOL

This appendix is part of the frozen evaluation rules.

Description format

One JSON object per clip, with *clip_id* and these fields. Systems may output more fields; only these are scored.

Field	Values
arousal, valence, dominance	Integers from 1 to 5. Arousal runs from low to high energy, valence from negative through neutral (3) to positive, and dominance from submissive through neither (3) to assertive.
emotion	One of neutral, happy, sad, angry, fearful, surprised or disgusted.
events	A list of non-speech events, each with type (laugh, giggle, sigh, breath, cough, filler or cry) and start and end in seconds.
timbre	A list of tags from breathy, creaky, nasal, hoarse, bright, dark, warm, thin and tense.
description	Free text in English, at most 60 words.

Event and timbre matching. Match events one to one within each type, pairing by largest overlap and requiring an intersection over union of at least 0.5. Reference events are those both listeners marked, matched the same way, with their boundaries averaged. Unmatched predictions are false positives, and unmatched references are false negatives. Timbre references are tags both listeners chose. Compute each F1 pooled over clips in each language, then average Hindi and Tamil.

Measured fields

Compute these on 16 kHz mono audio with librosa 0.11.0 and pyloudnorm 0.1.1, without normalizing amplitude:

Field	How it is computed	Tolerance
pitch_hz	Median F0 of voiced frames from librosa.pyin with fmin 65 Hz, fmax 500 Hz, frame_length 2048 and hop_length 160	±15%
rate	Unicode letters in the transcript divided by the clip's duration after librosa.effects.trim with top_db 30, frame_length 2048 and hop_length 512	±15%
loudness_lufs	Integrated loudness from pyloudnorm.Meter(16000)	±3 LU

A target field that can't be computed is left out for both systems; an output field that can't be computed fails.

File layout

For each language and split (describe, simulate):

- *inputs/manifest.csv: clip_id, group_id, and audio_path (describe) or text* (simulate).
- *inputs/targets.jsonl (simulate): clip_id, the first listener's Appendix E fields and* description, and the measured fields.
- *answers/ratings.jsonl: both listeners' records, each with clip_id and rater_id.*
- *answers/metadata.csv: clip_id, speaker_id, transcript, and the reference* audio path (simulate).

IDs and file names must not reveal labels.

Selection and bootstrap

Use the Appendix A procedures with these selections:

Selection	Candidates	m
t2_sets	Every other Track 2 team's set	3
t2_retrieval_hi, t2_retrieval_ta	Every describe group in the selected sets in that language	20 each
t2_target_<group_id>	The group's four clips	1

Selection	Candidates	m
t2_preference_hi, t2_preference_ta	Every simulate input in the selected sets in that language	20 each

If fewer candidates remain, use all of them. For confidence intervals, resample groups in each language, and keep the clips of each group together. Use identical draws for your system, round 0 and the control. Use the keys *t2_<measure>/<language>*, adding */<dimension>* for arousal, valence and dominance, and average Hindi and Tamil equally. Description match fixes the baseline value *b* at 0.5, like voice gain.

Trial records. The organizers record every retrieval and description-match trial: the target, the candidates or side order, audio hashes, a coded listener ID and the response.